2.2 Binding a Form to an Object

While this is an increase in comfort it is far from perfect. If `Person` has many attributes to edit then we would be repeating the name of the edited object many times. What we want to do is somehow bind a form to a model object, which is exactly what `form_for` does.

Assume we have a controller for dealing with articles `app/controllers/articles_controller.rb`:

```
def new
  @article = Article.new
end
```

The corresponding view `app/views/articles/new.html.erb` using `form_for` looks like this:

```
<%= form_for @article, url: {action: "create"}, html: {class:
"nifty_form"} do |f| %>
  <%= f.text_field :title %>
  <%= f.text_area :body, size: "60x12" %>
  <%= f.submit "Create" %>
<% end %>
```

There are a few things to note here:

- `@article` is the actual object being edited.
- There is a single hash of options. Routing options are passed in the `:url` hash, HTML options are passed in the `:html` hash. Also you can provide a `:namespace` option for your form to ensure uniqueness of id attributes on form elements. The namespace attribute will be prefixed with underscore on the generated HTML id.
- The `form_for` method yields a **form builder** object (the `f` variable).
- Methods to create form controls are called **on** the form builder object `f`.

The resulting HTML is:

```
<form accept-charset="UTF-8" action="/articles" method="post"
class="nifty_form">
  <input id="article_title" name="article[title]" type="text" />
  <textarea id="article_body" name="article[body]" cols="60"
rows="12"></textarea>
  <input name="commit" type="submit" value="Create" />
</form>
```

The name passed to `form_for` controls the key used in `params` to access the form's values. Here the name is `article` and so all the inputs have names of the form `article[attribute_name]`. Accordingly, in the `create` action `params[:article]` will be a hash with keys `:title` and `:body`. You can read more about the significance of input names in the parameter_names section.

The helper methods called on the form builder are identical to the model object helpers except that it is not necessary to specify which object is being edited since this is already managed by the form builder.

You can create a similar binding without actually creating `<form>` tags with the `fields_for` helper. This is useful for editing additional model objects with the same form. For example, if you had a `Person` model with an associated `ContactDetail` model, you could create a form for creating both like so:

```
<%= form_for @person, url: {action: "create"} do |person_form| %>
  <%= person_form.text_field :name %>
  <%= fields_for @person.contact_detail do |contact_details_form| %>
    <%= contact_details_form.text_field :phone_number %>
  <% end %>
<% end %>
```

which produces the following output:

```
<form accept-charset="UTF-8" action="/people" class="new_person" id="new_person"
method="post">
  <input id="person_name" name="person[name]" type="text" />
```